Implementing Distributed Transactions: 2PC State Machines, WAL & Recovery

Deep Technical Implementation Breakdown based on Arpit Bhayani's 2PC Engineering Lecture

IMPLEMENTATION REALITY & CRASH DURABILITY INVARIANT

Textbook distributed consensus assumes idealized nodes, but production systems suffer power cuts, process kills, and disk cache delays. Implementing Two-Phase Commit requires enforcing the **WAL Fsync Barrier**: a cohort must physically commit its `PREPARED` state, lock table entries, and undo/redo vectors to non-volatile storage via POSIX `fsync()` before emitting a network packet containing its vote. Without disk flush barriers, reboots cause state divergence and atomicity collapse.

1. ARCHITECTURE COMPONENTS & SYSTEM DECOMPOSITION

A production-ready Two-Phase Commit engine is composed of two discrete services operating across the network:

Coordinator Daemon Engine

- **Tx State Table:** Thread-safe concurrent map tracking active transaction IDs, registered cohorts, votes, and deadlines.
- **WAL Appender:** Sequential log writer flushing `START_2PC`, `GLOBAL_COMMIT`, and `END_2PC` records.
- **Deadline Watchdog:** Event loop enforcing prepare timeouts ($\tau_{prepare}$) to proactively trigger abort cascades.

Cohort Resource Manager

- **2PL Lock Manager:** Acquires exclusive write locks on primary keys; rejects concurrent transactions during voting.
- **Undo/Redo Store:** Maintains pre-images for rollback and post-images for commit durability.
- **Recovery Daemon:** Scans WAL records on process boot to resolve in-doubt transactions.

2. FINITE STATE MACHINE FORMALIZATION

The system couples two state machines synchronized through network RPCs and local disk events:

- **Coordinator FSM:** Transitions through `INIT` → `PREPARING` → `COMMITTING` (or `ABORTING`) → `TERMINATED`. The irreversible transition occurs when `GLOBAL_COMMIT` is physically synced to disk.
- **Cohort FSM:** Transitions from `READY` → `PREPARED` → `COMMITTED` (or `ABORTED`). Once in `PREPARED`, the cohort enters the "in-doubt" window and surrenders unilateral abort authority.

3. THE WRITE-AHEAD LOG (WAL) FLUSH CONTRACT

A standard `write()` syscall merely populates OS page cache. A crash before OS dirty pages are flushed leaves transactions lost. 2PC demands explicit `fsync()` barriers:

Role	Lifecycle Checkpoint	WAL Record Type	Must fsync Before Network Event
Cohort	Phase 1 Vote Decision	`PREPARED(TxID, Undo, Redo)`	Must fsync **before** transmitting `VOTE_COMMIT` across socket.
Coordinator	Phase 2 Decision	`GLOBAL_COMMIT(TxID, Cohorts)`	Must fsync **before** sending `GLOBAL_COMMIT` RPCs.
Coordinator	Phase 2 Abort	`GLOBAL_ABORT(TxID)`	Must fsync before returning error response to client.

4. CRASH RECOVERY PROTOCOL & REBOOT HANDLERS

When a server process dies (due to power failure, kernel panic, or OOM killer) and boots back up, its crash recovery loop reads the local WAL from byte 0 to the end of the file.

Cohort Recovery Rules

1. **Last Record is `INIT` or No Record:** Cohort crashed before voting. No lock or disk promises were made. Safely unlock resources and abort locally.
2. **Last Record is `COMMITTED` or `ABORTED`:** Transaction finished before crash. Nothing to reconcile.
3. **Last Record is `PREPARED` (In-Doubt Dilemma):** The cohort promised to commit, but does not know if the Coordinator committed or aborted.
   - Cohort must **re-acquire all row locks** to shield dirty data from concurrent clients.
   - Cohort sends an `INQUIRE_STATUS(TxID)` RPC to the Coordinator.
   - If Coordinator returns `GLOBAL_COMMIT`: apply redo log, write `COMMITTED`, release locks.
   - If Coordinator returns `GLOBAL_ABORT`: apply undo log, write `ABORTED`, release locks.

Coordinator Recovery Rules

1. **Last Record is `START_2PC` (No Decision Recorded):** The Coordinator died while collecting votes in Phase 1. It flushes `GLOBAL_ABORT` to disk and broadcasts abort RPCs to all cohorts.
2. **Last Record is `GLOBAL_COMMIT` (No `END_2PC`):** Transaction committed, but Coordinator crashed before receiving all ACKs. It re-broadcasts `GLOBAL_COMMIT` until all ACKs are collected, then appends `END_2PC`.

5. CONCURRENCY CONTROL & DISTRIBUTED DEADLOCK AVOIDANCE

When multiple coordinators execute 2PC transactions concurrently across the same cohorts, interlocking read/write dependencies can induce distributed deadlocks:

Canonical Resource Ordering

Every transaction must acquire partition and key locks in strict lexicographical order: $ ext{Key}_1 < ext{Key}_2 < \dots < ext{Key}_N$. This guarantees a Directed Acyclic Graph (DAG) for lock dependencies, making cycles impossible.

Wound-Wait Deadlock Prevention

Each transaction is tagged with a monotonic timestamp. If older transaction $T_{ ext{old}}$ requests a lock held by younger $T_{ ext{young}}$, $T_{ ext{young}}$ is wounded (aborted). If younger requests older, it waits.

6. PRODUCTION IMPLEMENTATION PATTERNS

Building high-performance 2PC systems requires addressing fundamental OS and network performance bottlenecks:

Group Commit Optimization

Each physical `fsync()` on an NVMe SSD takes ~1.5 ms, limiting single-threaded throughput to ~600 TPS. Group commit aggregates 50–200 concurrent prepare requests into a single continuous disk block write, scaling throughput beyond 25,000 TPS.

Idempotent RPC Handlers

Because network packets can be duplicated or delayed, Cohort `commit()` and `abort()` endpoints must be idempotent: applying an already-applied commit must return success immediately without error.

7. ARCHITECTURAL COMPARISON: TEXTBOOK 2PC VS. PRODUCTION 2PC

Dimension	Textbook Academic 2PC	Production Distributed 2PC (e.g. Spanner, CockroachDB)
Coordinator Durability	Single point of failure; crashes halt system	Coordinator is a 3-way Paxos/Raft replicated consensus group
Disk Logging	Abstract "log to stable storage"	Explicit POSIX `fsync()` barriers + Group Commit batching
Network Transport	Assumes reliable synchronous RPC	gRPC over HTTP/2, exponential backoff, circuit breaking
Locking Duration	Full duration of 2 RTTs + fsync	Optimistic Concurrency Control (OCC) + Multi-Version Concurrency (MVCC)

8. KEY ENGINEERING TAKEAWAYS

1. **Fsync Before Network Egress:** The golden rule of distributed transactions is that local state promises must hit durable disk media before the network interface emits the confirmation vote.
2. **Coordinator Replicated Consensuses:** Never deploy 2PC in production with a single, un-replicated coordinator. Run the coordinator logic inside a Raft or Paxos state machine.
3. **Deterministic Lock Sorting:** Prevent distributed deadlocks by sorting partition keys before issuing Prepare RPCs across shards.
4. **In-Doubt Recovery Daemon:** Cohorts must be self-healing: upon restart, they must actively resolve prepared transactions rather than leaving locks frozen indefinitely.